

Day 4: Optimization & Regularization

Training Stability, Initialization, and Preventing Overfitting

1. Batch Normalization (BatchNorm)

Training deep networks is difficult due to *Internal Covariate Shift*—as weights in early layers change, the distribution of inputs to later layers constantly shifts, forcing the network to continuously adapt to a moving target.

What it Normalizes

Batch Normalization stabilizes this by normalizing the intermediate feature maps. For a given mini-batch of size m , it calculates the mean and variance for **each channel** independently across the batch and spatial dimensions.

$$\begin{aligned}\mu &= (1 / m) \sum x_i \\ \sigma^2 &= (1 / m) \sum (x_i - \mu)^2 \\ x_{\text{hat}} &= (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} \\ y &= \gamma * x_{\text{hat}} + \beta\end{aligned}$$

Note the learnable parameters γ (scale) and β (shift). The network can learn to undo the normalization if the original distribution was actually optimal.

Where it Sits: Conv → BN → ReLU

It is almost universally placed **after** the Convolution and **before** the non-linear Activation (ReLU).

Mathematical Intuition: ReLU zeroes out all negative numbers. If the pre-activation distribution is

heavily skewed negative, most neurons die. BN centers the distribution at 0 (mean) with a variance of 1, ensuring exactly half the inputs pass through the ReLU, maximizing gradient flow.

Why it Stabilizes Training

- **Smoother Loss Landscape:** It fundamentally restructures the loss landscape, making gradients more predictive and less chaotic.
- **Higher Learning Rates:** Because the gradients are stable, we can use much larger learning rates without diverging.

2. Dropout: The Ensemble Intuition

Dropout is a powerful regularization technique designed to prevent neurons from co-adapting (i.e., relying on specific other neurons to fix their mistakes).

The Mechanism

During training, for every forward pass, we randomly "drop" (set to zero) a percentage of neurons (e.g., $p = 0.5$). If a neuron is dropped, it neither contributes to the forward pass nor receives updates during backpropagation.

Intuition as an Ensemble Method:

Because different neurons are dropped in every single batch, the architecture of the network is physically changing at every step. If you train for 10,000 steps, you are essentially training 10,000 slightly different, smaller sub-networks. At test time, turning Dropout off mathematically approximates averaging the predictions of this massive ensemble, which drastically reduces variance and overfitting.

Train vs. Eval Mode

Because we drop 50% of the neurons during training, the sum of the outputs entering the next layer is roughly halved. If we simply turned Dropout off during testing, the next layer would suddenly receive twice the signal amplitude, instantly destroying the predictions.

- **Training Phase:** Apply dropout mask. To compensate for the lost signal, modern frameworks use *Inverted Dropout*, scaling the active neurons up by $1 / (1 - p)$ immediately during training.
- **Eval Phase (`model.eval()`):** Dropout is completely turned off. Because we scaled the weights during training, no scaling is needed here, making inference as fast as possible.

3. Weight Initialization

Initializing a neural network properly is the difference between convergence in 10 epochs and a network that outputs `NaN` on step one.

Why Zero Initialization is Catastrophic

If you initialize all weights to **0** (or any identical constant), every neuron in a given layer will compute the exact same output. During backpropagation, they will all receive the exact same gradient. They will update identically. The network will mathematically never break symmetry, effectively acting as if each layer only has a single neuron.

Xavier / Glorot Initialization

Designed for symmetric activations like **Tanh** and **Sigmoid**. The goal is to keep the variance of the input identical to the variance of the output across a layer to prevent signals from exploding or vanishing.

```
W ~ Normal(0, Var)
Var(W) = 2 / (fan_in + fan_out)
```

Where `fan_in` is the number of input connections to the node, and `fan_out` is the number of output connections.

Kaiming / He Initialization

Xavier fails miserably for **ReLU**. Why? Because ReLU instantly zeroes out half of the distribution. This destroys half the variance. To compensate, Kaiming He introduced an initialization that doubles the variance.

```
Var(W) = 2 / fan_in
```

Rule of Thumb: Using ReLU? You MUST use Kaiming Initialization.

4. Optimizers: Navigating the Landscape

Optimizers dictate how the weights are updated using the computed gradients.

1. SGD + Momentum

Standard Stochastic Gradient Descent simply takes a step in the direction of the current gradient. This is noisy and gets stuck in ravines (bouncing wall-to-wall without moving forward).

Momentum introduces a velocity vector. It accumulates a moving average of past gradients. If the gradients consistently point in one direction, velocity builds up, pushing the optimizer through shallow local minima.

$$\begin{aligned}v_t &= \beta * v_{t-1} + (1 - \beta) * \nabla L \\W &= W - \alpha * v_t\end{aligned}$$

2. Adam (Adaptive Moment Estimation)

Adam tracks both the First Moment (mean of gradients, like Momentum) and the Second Moment (uncentered variance of gradients). It uses the variance to adapt the learning rate *per parameter*. If a weight has huge, chaotic gradients, Adam shrinks its learning rate. If a weight has tiny, consistent gradients, Adam boosts it.

When to prefer: Almost always for initial prototyping, NLP, and complex architectures. It converges incredibly fast with default parameters.

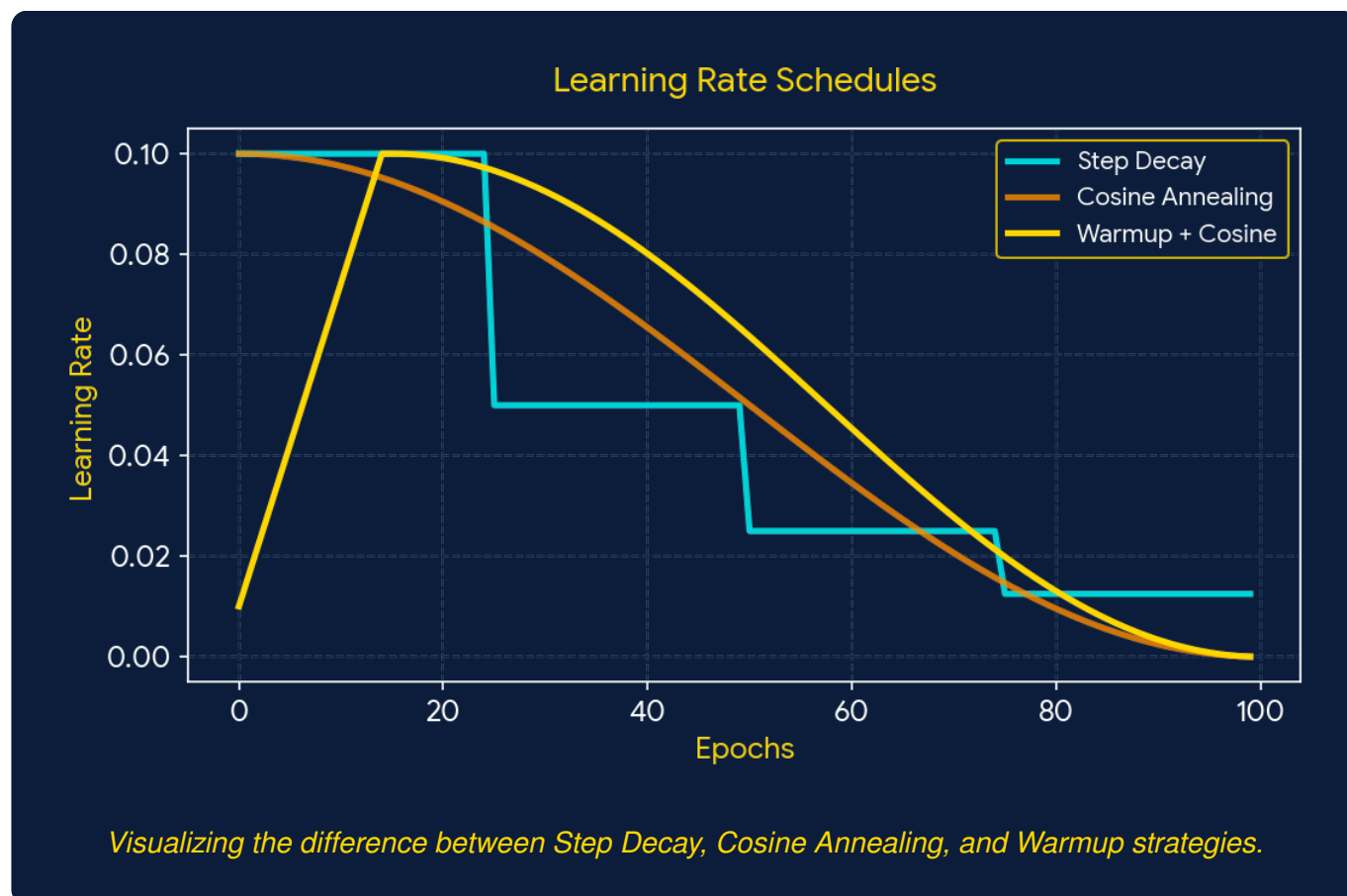
3. AdamW (Adam with Weight Decay)

Standard Adam implements L2 Regularization (Weight Decay) incorrectly due to how the adaptive denominator interacts with the decay penalty. AdamW **decouples** the weight decay from the gradient update, applying it directly to the weights.

When to prefer: For state-of-the-art vision models (Vision Transformers) or anytime you need aggressive regularization to prevent overfitting.

5. Learning Rate Schedules

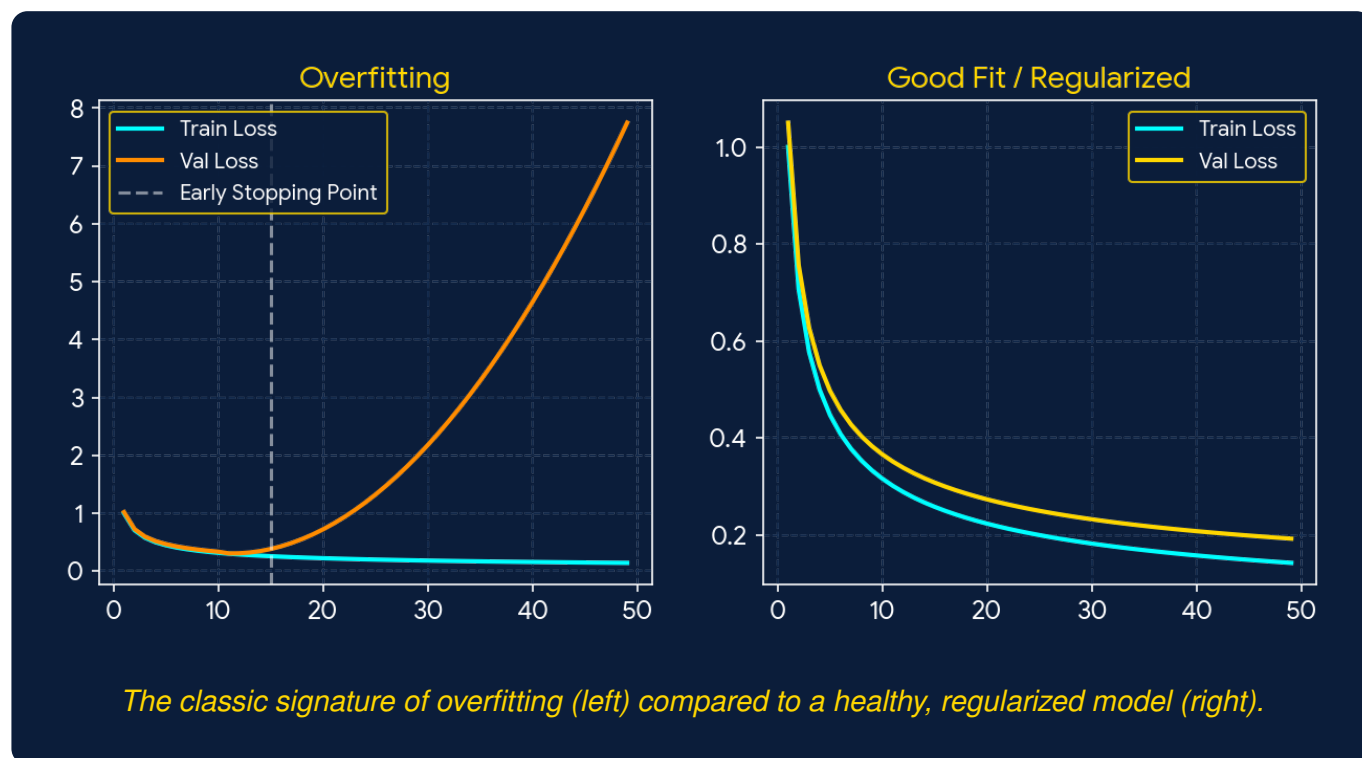
A static learning rate is suboptimal. High LRs learn fast but bounce around the minimum. Low LRs settle into the minimum but take forever to get there. Schedules dynamically adjust the LR during training.



- **Step Decay:** Multiplies the LR by a factor (e.g., 0.1) every N epochs. Creates a "staircase" effect. Simple, but the sudden drops can shock the optimizer.
- **Cosine Annealing:** Smoothly curves the LR down following a cosine wave. It starts dropping slowly, accelerates in the middle, and gently settles at the end.
- **Warmup:** Starts the LR very close to 0 and ramps it up linearly over the first few epochs. *Why?* At epoch 0, weights are random, and gradients are massive. A high LR instantly causes the weights to explode. Warmup stabilizes the initial chaotic phase before handing over to a decay schedule.

6. Diagnosing Fit: Overfitting vs. Underfitting

Your loss curves are the ultimate diagnostic tool for the health of your network.



Overfitting (High Variance)

The model has memorized the training data noise instead of learning the underlying distribution.

- **Signature:** Training loss approaches 0. Validation loss hits a minimum, then begins to aggressively **rise**.
- **Solution:** More data, Data Augmentation, Dropout, Weight Decay (AdamW), Early Stopping (halting training at the dotted line).

Underfitting (High Bias)

The model is too simple to capture the complexity of the data.

- **Signature:** Both Training and Validation loss remain unacceptably high and flatline early.

- **Solution:** Increase model capacity (more layers/neurons), reduce regularization (lower Dropout/Weight Decay), train longer.

7. Data Augmentation as Regularization

The best way to prevent overfitting is to get more data. When that is impossible, we artificially synthesize new data by perturbing the existing images. This expands the data manifold and forces the network to learn invariant features rather than memorizing exact pixel layouts.

Common Techniques (Vision)

- **Random Crop:** Pad the image, then randomly crop it back to the original size. Teaches the network that an object is still the object even if it is not perfectly centered.
- **Horizontal Flip:** Randomly mirroring the image (e.g., $p = 0.5$). A cat facing left is identical in meaning to a cat facing right. (Do not use this for tasks where direction matters, like digit recognition—a flipped '3' is an 'E').
- **Color Jitter:** Randomly shifting the brightness, contrast, saturation, and hue. Teaches the network to recognize the geometric structure of a car regardless of whether it is pictured on a sunny day or in deep shadow.

Mathematical View: Data augmentation acts as a continuous, infinite injection of noise. Because the network never sees the exact same image matrix twice, it is mathematically impossible for it to perfectly memorize the dataset, effectively capping the Training Accuracy and pulling the Validation Accuracy up alongside it.